By: Shoumik, Moneba, Judy, Ekam & Abrar

Group Project: Team AB
FreshRoute System Design Document

Team Members:

Name	Email	Student ID
Moneba Hanifi	moneba@my.yorku.ca	219721810
Shoumik Ahmed	shoumik@my.yorku.ca	219881598
Judy El Ghargomi	jelg21@my.yorku.ca	220227237
Ekambir Momi	ekam727@my.yorku.ca	219424407
Abrar Jawad Tarafder	abrart03@my.yorku.ca	219453497

3311 Section M Term W

Class Instructor: Ilir Dema

March 4, 2026

Cover Page 1
Table Of Contents 2

Chapter 1: Document Overview

Introduction 3

Chapter 2: System Overview and Design

Architecture Diagram 4
Component Design CRC Cards 5
Assumption and Future Work 9

Chapter 1: Document Overview

INTRODUCTION

This document describes the system design for FreshRoute, a web-based grocery platform focused on organic and ethically sourced food products. The goal of this document is to outline the system's overall structure, the technologies used, and the responsibilities of the major components.

FreshRoute follows a MERN stack architecture, comprising a React frontend, a Node.js and Express backend, and a MongoDB database accessed via Mongoose. Communication between the frontend and backend occurs through RESTful APIs using HTTP and JSON, with Cross-Origin Resource Sharing (CORS) enabled for development.

The backend follows a Model–View–Controller (MVC) design pattern. Express routers define the API endpoints, controllers handle incoming requests and application logic, and models define the MongoDB schemas and database interactions. The backend application is configured in app.js, while server.js serves as the entry point, connecting to the database and starting the HTTP server.

The frontend is implemented as a component-based React single-page application using Vite. The root component (App.jsx) manages page routing and loads page components, including Landing, Organic, EthicallyRaised, Recipes, and Cart. These pages share common interface elements such as the navigation bar (AppNavbar).

Data currently flows from the frontend to the backend through API calls such as /api/products, which return JSON responses. The frontend state is currently managed locally within components without a global state store.

During development, the backend server runs on port 4000 and the frontend development server runs on port 5173. Environment variables, such as the MongoDB connection string, are configured in a .env file.

Chapter 2: System Overview

Architecture Diagram

COMPONENT DESIGN

CRC CARDS

Frontend Components: React Components

AppNavbar	Frontend
Parent Class:	React component
Subclasses:	None
Responsibilities: - Nav links to main sections. - Search/location input. - Cart badge + welcome text.	
Collaborators: - All pages (shared navbar)	

Landing	Frontend
Parent Class:	React component
Subclasses:	None
Responsibilities: - Hero + value props. - CTA to shop (navigate into store pages).	
Collaborators: - AppNavbar	

Organic

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:
- Tabbed produce grid.
- Client-side search/filter.
- Product detail modal + CTAs.

Collaborators:
- AppNavbar

EthicallyRaised

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:
- Tabbed protein grid.
- Client-side search/filter + modal.

Collaborators:
- AppNavbar

Recipes

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:
- Tabbed recipe catalog + search.
- Modal showing scaled servings + ingredients.
- CTA to add servings to cart (UI only for now).

Collaborators:
- AppNavbar

Cart

Frontend

Parent Class: React component
Subclasses: None

Responsibilities:
- Cart view shell (placeholder in Sprint 1).

Collaborators:
- AppNavbar

Backend Components: Node / Express / MongoDB Components

server.js

Backend

Parent Class: None
Subclasses: None

Responsibilities:
- Load env, connect DB, start HTTP server.
- Fail fast if MONGO_URI missing/connection fails.

Collaborators:
- connectDB
- app (Express app)

productRoutes

Backend

Parent Class: Express Router
Subclasses: None

Responsibilities:
- Map /api/products to controller handlers.

Collaborators:
- ProductController

app.js (Express app)

Backend

Parent Class: Express application
Subclasses: None

Responsibilities:
- Configure middleware (JSON parsing, CORS).
- Provide health/root routes.
- Mount product routes.

Collaborators:
- productRoutes

productRoutes

Backend

Parent Class: Express Router
Subclasses: None

Responsibilities:
- Map /api/products to controller handlers.

Collaborators:
- ProductController

ProductController

Backend

Parent Class: None

Subclasses: None

Responsibilities:

- Handle product CRUD (Sprint 1: create + list).
- Return proper HTTP codes (201 success, 400 validation).

Collaborators:

- Product model

Product (model)

Backend

Parent Class: Mongoose Model

Subclasses: None

Responsibilities:

- Define schema + validation (name, price, category, inStock, timestamps).
- Expose queries for controllers (list/sort by createdAt).

Collaborators:

- MongoDB
- ProductController

connectDB

Backend

Parent Class: None

Subclasses: None

Responsibilities:

- Establish Mongo connection and surface errors.

Collaborators:

- Mongoose
- server.js

ASSUMPTION AND FUTURE WORK

The FreshRoute system assumes a development environment that supports Node.js version 18 or later with ES module support enabled. The project relies on npm for dependency management and development scripts, while the frontend uses the Vite development server to build and run the React application. The MongoDB Community Server must be reachable through the connection string defined in the `MONGO_URI` environment variable, and Mongoose automatically creates collections and manages schema validation when the application interacts with the database.

The system is designed to run on any operating system that supports Node.js and MongoDB, including Windows, macOS, and Linux. Network communication between the frontend and backend occurs over HTTP, with CORS enabled during development to allow requests from the frontend development server. Security features such as HTTPS and advanced network configurations are not currently implemented.

Error handling is currently implemented at a basic level. Validation errors return HTTP 400 responses with an error message, while database connection failures cause the server process to terminate after logging the error. Requests to undefined routes return standard Express 404 responses. At the moment, frontend error feedback is limited to console logging rather than user-facing notifications.

From a performance perspective, the system currently runs as a single Node.js process without clustering or load balancing. This configuration is sufficient for course-scale development and testing, but may require enhancements such as pagination, caching, or rate limiting for larger workloads.

Security features are also limited in the current implementation. User authentication and authorization are not yet implemented, and input validation primarily relies on Mongoose schema validation. Future development may introduce authentication mechanisms, additional validation middleware, and security tools such as Helmet to improve system protection.

Future improvements to the FreshRoute system may include implementing full backend integration for frontend pages, introducing user accounts and authentication, improving error reporting and user feedback within the interface, and adding global state management to better manage application data across components.